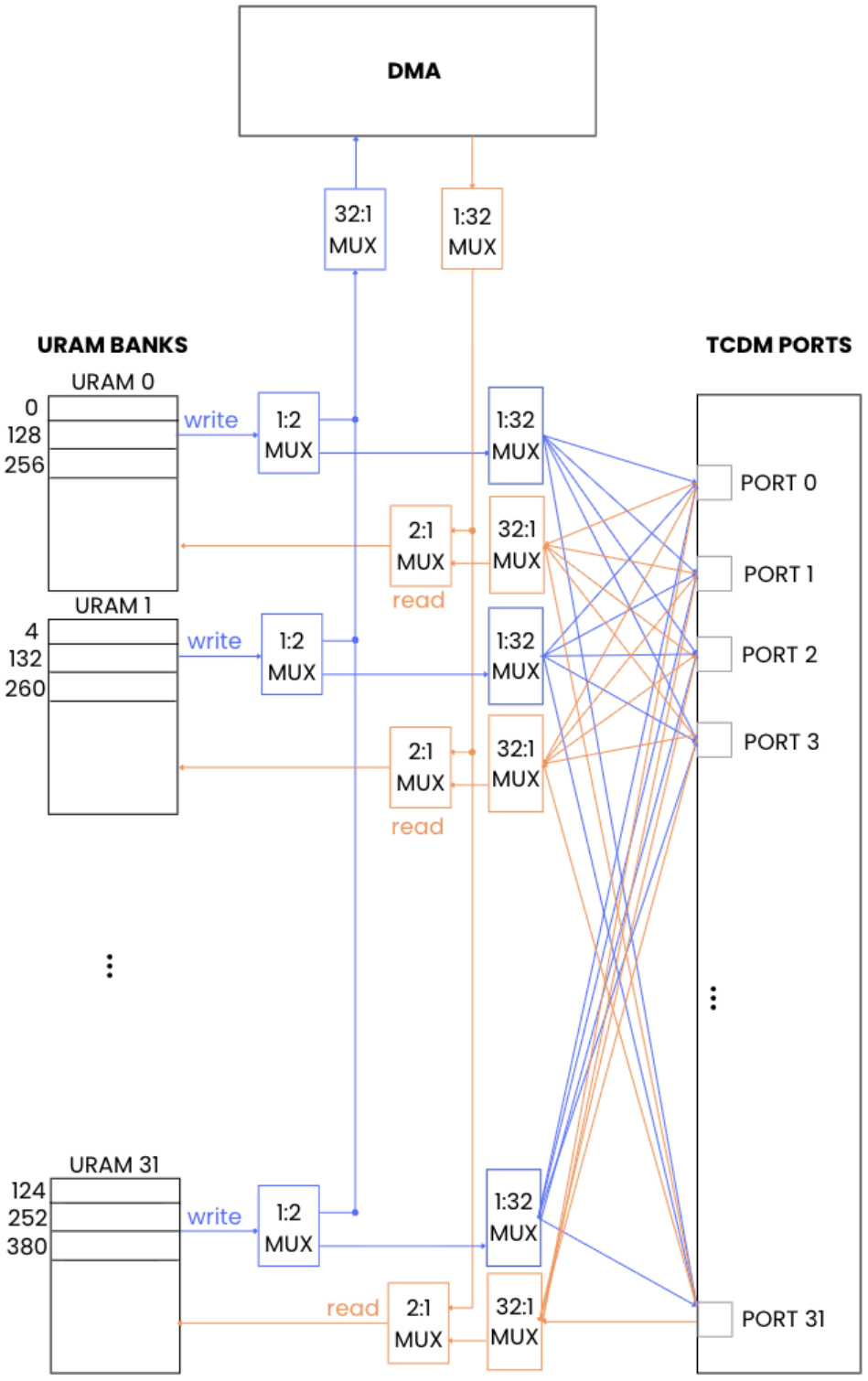
ITA-URAM-DMA CONTROLLER SPECIFICATION

This document specifies the architecture and behavior of a synthesizable URAM memory controller designed for the ITA accelerator platform on an FPGA. The architecture implements a 32-bank interleaved memory system, providing two primary access modes: a high-performance, 32-port TCDM interface for the ITA accelerator, and a streaming DMA interface for data loading and unloading. This dual-mode design is optimized to handle both the concurrent access patterns of the ITA and efficient bulk data transfers.

Architecture



The memory system is constructed from 32 independent URAM instances, each acting as a memory bank to be compatible with the 1024 bit data transfer width of ITA. The controller provides two arbitrated master interfaces to these banks: the 32-port TCDM interface and the single-port DMA interface.

2.1. Key Components

- **TCDM Ports (32):** The parallel interface through which the ITA accelerator requests read and write operations when dma_mode_i is low. Each port (tcdm[0] to tcdm[31]) is an hci_core_intf bundle.
- **DMA Port (1):** A single, high-throughput streaming interface for loading data into the URAMs from an external source (e.g., DDR memory) and reading results back out when dma_mode_i is high.
- **URAM Banks (32):** The physical memory storage, implemented using xpm_memory_tdpram primitives configured as URAMs. Each bank has a read port (Port A) and a write port (Port B).
- **Arbitration MUX:** A top-level multiplexer, controlled by the dma_mode_i signal, that selects whether the URAM banks are controlled by the ITA's TCDM interface or the DMA interface in any given cycle.
- **ITA Crossbar:** A network of multiplexers that routes requests from the 32 TCDM ports to the appropriate URAM banks when the system is in ITA mode.
- **DMA Demultiplexer:** Logic that routes a single DMA request to the correct target URAM bank based on the address when the system is in DMA mode.

TCDM Signals:

This mode is active when dma_mode_i is 0.

tcdm[i].req	Input	1	Request valid from TCDM Port i.
tcdm[i].wen	Input	1	Write enable from Port i. 0=Write, f=Read.
tcdm[i].add	Input	32	Byte address for the memory operation.
tcdm[i].data	Input	32	Data to be written for a write operation.
tcdm[i].be	Input	4	Byte enable for a write operation. Always has the value 0 or 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0, f,f,0,0,0,0,0,0,0,0,0,0,0,0,0,0,f,f
tcdm[i].gnt	Output	1	Grant signal to TCDM Port i. Pulsed high for one cycle when a request is accepted.
tcdm[i].r_data	Output	32	Data returned for a read operation.
tcdm[i].r_valid	Output	1	Asserted for one cycle when r_data is valid.

3.2. Memory Mapping and Address Decoding
The controller implements a memory interleaving scheme to distribute sequential addresses across the 32 URAM banks. This maximizes parallelism for the ITA's access patterns.
Given a 32-bit byte address tcdm.add:

- **Bits [1:0]:** Byte offset within a 32-bit word. These bits are ignored for bank and word selection.
- **Bits [6:2]: Bank Index**. These 5 bits (BANK_SEL_W = 5) directly select one of the 32 URAM banks. The bank index is (address / 4) % 32.
- **Bits [31:7]: Local Word Address**. These upper bits form the address within the selected URAM bank. The local address is address / 128.

This results in the following mapping of 32-bit words:

- Word at Address 0x0000 -> Bank 0, Local Address 0
- Word at Address 0x0004 -> Bank 1, Local Address 0
- ...
- Word at Address 0x007C -> Bank 31, Local Address 0
- Word at Address 0x0080 -> Bank 0, Local Address 1

Example Encoding Scheme:
For a byte address A**:**

```
        A[31:7]                 A[6:2]            A[1:0]
   <----------------->        | <---------------> |    <------>
Local Word Address    Bank Index (0-31)    Byte Sel (ignored)
(Sent to URAM Addr)   (Selects URAM #)
```

3.3. Operation and Arbitration
The controller utilizes the dual-port nature of the URAMs to separate read and write operations.

Handshake:
The tcdm.gnt signal is asserted combinationally in the same cycle as tcdm.req. This simple handshake indicates the request has been received by the controller but does not guarantee **execution** if a bank conflict occurs.

Bank Conflict Arbitration:
The crossbar logic evaluates all incoming requests simultaneously. If multiple TCDM ports target the same URAM bank in the same clock cycle, a fixed-priority scheme resolves the conflict: tcdm[0] has the highest priority, and tcdm[31] has the lowest. Only the request from the highest-priority port is passed to the URAM bank. Lower-priority requests targeting the same bank in the same cycle are ignored for that cycle. It is the responsibility of the master (ITA) to re-issue any requests that were not serviced due to a bank conflict.

Write Operations:
A TCDM port j initiates a write by asserting req with wen=0. If it wins arbitration for its target bank i, its add, data, and be signals are routed through the write crossbar to the write port (Port B) of URAM bank i.

Read Operations:
A TCDM port j initiates a read by asserting req with wen=1. If it wins arbitration for its target bank i, its add signal is routed to the read port (Port A) of URAM bank i.

Read Data Latency:
Read data has a fixed latency of one clock cycle from the request.

- **Cycle 0:** A port tcdm[j] asserts req. If it wins arbitration, its address is sent to the target URAM bank i. The controller internally registers that port j is expecting data from bank i.
- **Cycle 1:** The URAM outputs the requested data. The read return logic uses the registered bank index (i) to select the correct data from the 32 bank outputs and routes it back to the original requester on tcdm[j].r_data. The tcdm[j].r_valid signal is asserted in this cycle.

4. Streaming DMA Interface (DMA Mode)

This mode is active when dma_mode_i is 1. It provides a simple, single-port AXI-Stream-like interface for bulk data transfers.

4.1. DMA Signals

Signal	Direction	Description
dma_we_i	Input	Selects transfer direction. 1=Write to URAM, 0=Read from URAM.
dma_addr_valid_i	Input	Indicates dma_addr_i is valid.
dma_addr_i	Input	The byte address for the current transfer word.
dma_addr_ready_o	Output	Indicates the controller is ready for a new address.
dma_wdata_valid_i	Input	Indicates dma_wdata_i is valid for a write operation.
dma_wdata_i	Input	The 32-bit data word to be written to URAM.
dma_wdata_ready_o	Output	Indicates the controller is ready for a new data word.
dma_rdata_valid_o	Output	Indicates dma_rdata_o is valid for a read operation.
dma_rdata_o	Output	The 32-bit data word read from URAM.
dma_rdata_ready_i	Input	Indicates the external system is ready for read data.

4.2. Operation and Handshake

The DMA interface uses a coupled handshake mechanism.

Write Operation (dma_we_i = 1):
A write to a URAM bank occurs only when both dma_addr_valid_i and dma_wdata_valid_i are asserted in the same cycle. The dma_addr_i is decoded to select a target bank, and the dma_wdata_i is written to it. The controller signals its readiness via dma_addr_ready_o and dma_wdata_ready_o.

Read Operation (dma_we_i = 0):
A read is initiated when dma_addr_valid_i is asserted and the controller is ready (dma_addr_ready_o is high). The dma_rdata_valid_o signal will be asserted **and held high** until the transaction is accepted by the destination (dma_rdata_ready_i is high). This implements a standard AXI-Stream handshake.

- Cycle 0: The read address is accepted and sent to the target URAM bank.
- Cycle 1: The dma_rdata_valid_o signal is asserted for one cycle, and the corresponding dma_rdata_o is driven with the data from the URAM.